

LAPORAN AKHIR PROJECT BASED LEARNING (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Tahun Akademik 2025/2026

Dosen: Dr. Shelve Nidya Neyman

JUDUL PROYEK:

APLIKASI BUKU TELEPON PINTAR

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

Aulia Rezki Rahman - J0403251009

Ivan Farel Aditya - J0403251087

Raffan Fatih Zulfan - J0403251091

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan laporan akhir Project-Based Learning (PBL) untuk mata kuliah Algoritma & Struktur Data (TPL2106) dengan baik. Proyek aplikasi "Buku Telepon Pintar" ini disusun untuk menerapkan berbagai konsep struktur data dan algoritma yang telah diajarkan ke dalam sebuah aplikasi berbasis konsol (CLI).

Kami mengucapkan terima kasih yang sebesar-besarnya kepada Dr. Shelve Nidya Neyman selaku dosen pengampu yang telah memberikan bimbingan dan arahan selama proses perkuliahan dan pengerjaan proyek ini. Kami menyadari bahwa proyek ini masih jauh dari sempurna, oleh karena itu kami mengharapkan kritik dan saran yang membangun untuk pengembangan selanjutnya.

DAFTAR ISI

- BAB I. PENDAHULUAN
- BAB II. ANALISIS DAN PERANCANGAN
- BAB III. IMPLEMENTASI PROGRAM
- BAB IV. PENGUJIAN DAN ANALISIS
- BAB V. KENDALA DAN SOLUSI
- BAB VI. PEMBAGIAN TUGAS ANGGOTA
- BAB VII. KESIMPULAN DAN SARAN
- DAFTAR PUSTAKA
- LAMPIRAN

BAB I. PENDAHULUAN

1.1 Latar Belakang

Dalam kehidupan sehari-hari, data kontak seperti nama dan nomor telepon menjadi informasi krusial yang harus dikelola dengan efisien. Penyimpanan kontak secara manual seringkali menyulitkan dalam proses pencarian dan rentan terhadap kehilangan data. Oleh karena itu, proyek ini dibuat untuk membantu pengelolaan kontak telepon secara terstruktur dan digital menggunakan aplikasi CLI. Sistem yang dirancang mengimplementasikan struktur data *Dictionary* untuk memastikan operasi *Create, Read, Update, Delete* (CRUD) berjalan cepat. Program juga mengaplikasikan algoritma pencarian (*Sequential Search*) dan pengurutan (*Quick Sort*), serta mampu menyimpan data secara permanen menggunakan penanganan berkas (*file handling*) tipe .txt agar data tetap tersedia meski program telah ditutup.

1.2 Tujuan Proyek

1. Membuat aplikasi manajemen kontak (Buku Telepon) berbasis terminal (CLI) menggunakan bahasa pemrograman Python.
2. Mengimplementasikan struktur data *Dictionary* untuk optimalisasi penyimpanan di memori utama.
3. Menerapkan algoritma pencarian teks tak-sensitif (*case-insensitive substring search*) dan algoritma pengurutan *Quick Sort* secara hierarkis dan rekursif.
4. Menerapkan konsep *file handling* untuk menyimpan *state* memori ke dalam bentuk file TXT sehingga data bersifat permanen dan persisten.

1.3 Ruang Lingkup

- **Platform:** Command Line Interface (CLI) / Konsol terminal.
- **Bahasa:** Python.
- **Penyimpanan:** File teks (kontak.txt) yang disimpan secara otomatis / manual dalam satu

direktori.

- **Fitur Operasional:** CRUD (Tambah, Lihat, Update, Hapus).
- **Fitur Pencarian:** *Sequential Search* berdasarkan nama atau nomor.
- **Fitur Pengurutan:** *Quick Sort* secara alfabetis (A-Z) pada saat data akan disimpan.

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Aplikasi "Buku Telepon Pintar" ini merupakan program berbasis teks (CLI) yang berjalan terus-menerus menggunakan *looping* tanpa batas. Sistem memuat seluruh data dari kontak.txt ke dalam memori utama (berupa *Dictionary*) pada saat *start-up*. Pengguna dapat memilih 7 menu interaktif: menambah kontak, melihat daftar kontak, mencari spesifik kontak, mengubah nama/nomor, menghapus kontak, menyimpan data secara manual ke file, atau keluar. Data akan otomatis diurutkan dengan algoritma *Quick Sort* setiap kali pengguna melakukan penyimpanan permanen.

2.2 Struktur Data yang Digunakan

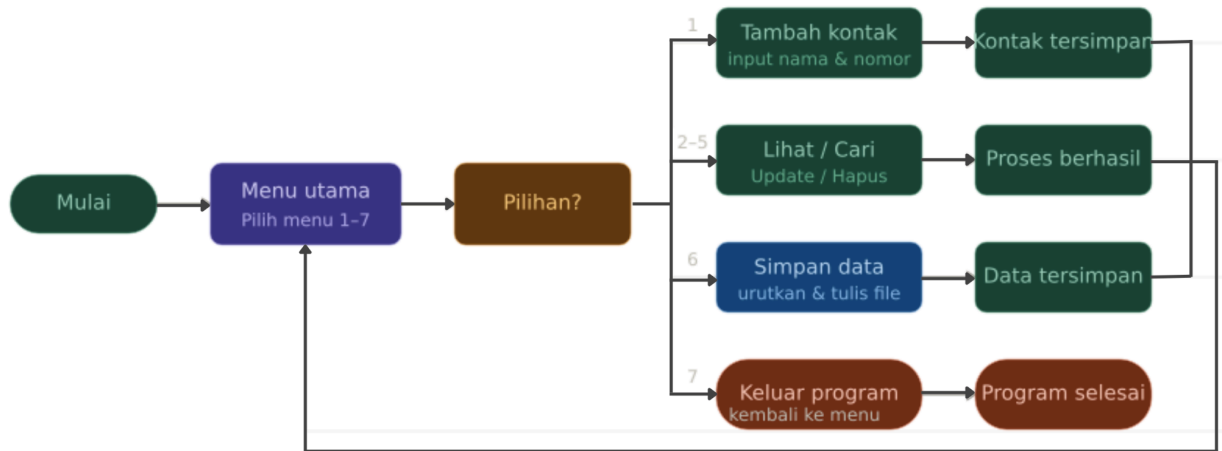
Struktur Data	Fungsi	Alasan Pemilihan
Dictionary (Hash Map)	Menyimpan data utama kontak secara presisten di memori komputer dengan struktur pasangan <i>Key</i> (Nama) dan <i>Value</i> (Nomor Telepon).	Akses data untuk pencarian <i>key</i> , penambahan, dan penghapusan sangat cepat dengan kompleksitas waktu rata-rata $O(1)$. Selain itu, <i>key</i> pada dictionary secara inheren mencegah duplikasi nama kontak yang sama.
List of Tuples	Wadah konversi (<i>casting</i>) data sementara sebelum	Tipe data <i>Dictionary</i> pada Python bukan tipe data

Struktur Data	Fungsi	Alasan Pemilihan
	dilakukan proses <i>sorting</i> .	terurut (indeks tidak absolut). Agar dapat diproses oleh algoritma <i>Quick Sort</i> , elemen <i>Dictionary</i> harus dikonversi menjadi <i>List</i> berisikan <i>Tuple</i> ('Nama', 'Nomor').

2.3 Algoritma yang Digunakan

Kategori	Algoritma	Alasan Penggunaan
Searching	<i>Sequential Search</i> (Pencarian Substring)	Pengguna sering kali hanya mengingat sebagian huruf (misal mencari "Budi" dengan mengetik "bud") atau sebagian nomor. <i>Sequential search</i> yang dikombinasikan dengan metode <code>.lower()</code> memastikan akurasi kecocokan substring secara utuh pada semua elemen.
Sorting	<i>Quick Sort</i>	Sangat efisien untuk menangani sekumpulan string atau daftar nama. Algoritma ini membagi data dengan sistem <i>pivot / partition</i> secara rekursif, menghasilkan performa kompleksitas waktu rata-rata yang optimal yaitu $O(n \log n)$.

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

Seluruh program berjalan melalui interaksi beberapa fungsi di dalam satu file skrip Python utama, dengan struktur sebagai berikut:

- kelompok8.py : File utama berisikan antarmuka dan logika program (CRUD, Sorting, Main Loop).
- kontak.txt : File teks datar (*plain-text file*) yang dihasilkan otomatis oleh program sebagai penyimpanan permanen berformat Comma-Separated (Nama,Nomor).

BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

Menu utama dibuat menggunakan fungsi main() dengan blok while True. Terdapat 7 opsi yang bisa dipilih.

```
=== DATA BUKU TELEPON ===
1. Tambah Kontak
2. Lihat Kontak
3. Cari Kontak
4. Update Kontak
5. Hapus Kontak
6. Simpan Data Manual
7. Keluar
Pilih menu (1-7): █
```

3.2 Fitur Create

Fungsi `tambah_kontak()` meminta input "Nama" dan "Nomor". Nama distandarisasi kapitalisasinya menggunakan `.title()`, kemudian dicek melalui `if` nama in daftar. Jika valid, program menetapkan `daftar[nama] = nomor`.

```
Pilih menu (1-7): 1
--- TAMBAH KONTAK BARU ---
Masukkan Nama: budi
Masukkan Nomor Telepon: 081273893349
Kontak 'Budi' berhasil ditambah (Pilih menu 6 untuk simpan permanen!).
```

```
Pilih menu (1-7): 1
--- TAMBAH KONTAK BARU ---
Masukkan Nama: raffan
Eitsss! Kontak dengan nama 'Raffan' sudah ada.
```

3.3 Fitur Read

Fungsi `lihat_kontak()` melakukan iterasi sederhana pada `daftar.items()` dan menampilkannya dengan rapi (*string formatting*) di terminal. Jika dictionary kosong, akan muncul peringatan "Buku telepon masih kosong".

```
Pilih menu (1-7): 2

--- DAFTAR KONTAK ---
Buku telepon masih kosong.
```

```
Pilih menu (1-7): 2

--- DAFTAR KONTAK ---
Nama: Aulia      | Nomor: 08413393749
Nama: Ivan       | Nomor: 0863548992
Nama: Raffan     | Nomor: 08123455678
Nama: Raihan     | Nomor: 0824832798203
Nama: Raxen      | Nomor: 0863293723932
Nama: Budi       | Nomor: 081273893349
```

3.4 Fitur Update

Fungsi `update_kontak()` memberikan fleksibilitas mengubah Nama atau Nomor. Karena sifat *key* pada *dictionary* yang *immutable* (tidak bisa diganti langsung), mengubah nama diimplementasikan dengan: menyimpan nomor (value), menghapus key lama dengan fungsi `del`, lalu membuat pasangan key-value baru.


```
Pilih menu (1-7): 4
--- UPDATE KONTAK ---
Masukkan Nama kontak yang ingin diubah: raihan
1. Ubah Nama saja
2. Ubah Nomor saja
Pilih bagian yang ingin diubah (1/2): 1
Masukkan Nama Baru: randy
Nama 'Raihan' berhasil diubah menjadi 'Randy'.
```

3.5 Fitur Delete

Fungsi hapus_kontak() memvalidasi eksistensi data terlebih dahulu. Jika ditemukan, elemen akan dibuang dari memori menggunakan sintaks `del daftar[nama]`.

```
Pilih menu (1-7): 5
--- HAPUS KONTAK ---
Masukkan Nama kontak yang ingin dihapus: raxen
Kontak 'Raxen' dihapus dari memori (Pilih menu 6 untuk simpan permanen!).
```

3.6 Fitur Searching

Algoritma yang digunakan adalah *Sequential Search* dengan metode *substring matching*. Pada fungsi `cari_kontak()`, sistem melakukan perulangan ke setiap iterasi `daftar.items()`. Kondisi pencariannya menggunakan `if kata_kunci.lower() in nama.lower()` untuk memastikan data dapat ditemukan tanpa terkendala penulisan huruf besar atau kecil (*case-insensitive*).

```
Pilih menu (1-7): 3
--- CARI KONTAK ---
1. Cari berdasarkan Nama
2. Cari berdasarkan Nomor
Pilih metode pencarian (1/2): 1
Masukkan Nama yang dicari: ra
Ketemu! -> Nama: Raffan      | Nomor: 08123455678
Ketemu! -> Nama: Raihan     | Nomor: 0824832798203
Ketemu! -> Nama: Raxen      | Nomor: 0863293723932
```

3.7 Fitur Sorting

Algoritma yang digunakan adalah *Quick Sort* murni secara rekursif melalui fungsi `quickSort()`, `quickSortHelper()`, dan `partition()`. Acuan pivot yang digunakan adalah indeks pertama dari partisi, dengan membandingkan nilai string alfabet pada elemen `data[first][0]`. Fungsi ini dijalankan sesaat sebelum menimpa dan menulis file `kontak.txt`.

```
Pilih menu (1-7): 6
Mantap! Data berhasil diurutkan (A-Z) dan disimpan permanen ke kontak.txt.
```

```
Pilih menu (1-7): 2
--- DAFTAR KONTAK ---
Nama: Aulia      | Nomor: 08413393749
Nama: Budi       | Nomor: 081273893349
Nama: Ivan       | Nomor: 0863548992
Nama: Raffan     | Nomor: 08123455678
Nama: Randy      | Nomor: 0824832798203
```

3.8 Penyimpanan Data (File Handling)

- **Format File:** Menggunakan ekstensi .txt dengan koma (,) sebagai pembatas kolom (separator).
- **Membaca File:** Dijalankan pada awal program oleh fungsi muat_data(). Program menggunakan library os.path.exists(). Jika file ada, program melakukan pembacaan with open(NAMA_FILE, 'r') dan memisahkan data dengan line.strip().split(',', 1).
- **Menyimpan File:** Dijalankan oleh simpan_data() dengan mode tulis tanpa with open(NAMA_FILE, 'w') setelah data diurutkan melalui Quick Sort.

```
≡ kontak.txt

1   Aulia,08413393749
2   Budi,081273893349
3   Ivan,0863548992
4   Raffan,08123455678
5   Randy,0824832798203
6
```

3.9 Validasi Input dan Error Handling

Skenario Kasus Error	Penanganan (Error Handling)
Duplikasi Nama pada tambah_kontak	Program memunculkan pesan "Eitsss! Kontak dengan nama ... sudah ada" dan menggagalkan input.

Skenario Kasus Error	Penanganan (Error Handling)
File belum tersedia saat eksekusi awal	Fungsi muat_data akan mendeteksi (dengan <code>os.path.exists</code>) lalu mengembalikan <i>dictionary</i> kosong sehingga program tidak <i>*crash*</i> atau <i>*error*</i> .
Pilihan input angka pada menu tidak valid	Blok <code>else</code> : menangkap input di luar "1-7" dan memunculkan peringatan "Pilihan tidak valid! Silakan pilih angka 1-7".
Nama target pada menu Update / Delete tidak ada	Program menampilkan pesan "Kontak tidak ditemukan" karena lolos validasi <code>if</code> nama in daftar:.

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

No	Skenario	Input	Ekspektasi Output	Hasil
1	Tambah Kontak Baru	Nama: "Asep", No: "0811"	Kontak berhasil masuk ke memori daftar	Berhasil
2	Duplikasi Kontak	Nama: "Asep", No: "0822"	Peringatan bahwa data "Asep" sudah ada. Data 0822 ditolak.	Berhasil
3	Edit Nama Kontak	Nama Lama: "Asep", Baru: "Bambang"	Key Asep hilang, muncul key Bambang bernomor sama	Berhasil

No	Skenario	Input	Ekspektasi Output	Hasil
4	Hapus Kontak	Nama: "Bambang"	Data "Bambang" dihapus total dari memori	Berhasil
5	Searching Substring	Metode: Nama, Keyword: "bam"	Menampilkan detail data "Bambang" (jika blm dihapus)	Berhasil
6	Sorting Data dan Penyimpanan	Pilih Menu 6	Data di list terurut (A-Z) dan muncul tulisan sukses tersimpan permanen ke kontak.txt	Berhasil

4.2 Kelebihan Sistem

1. **Sangat Cepat di Memori Utama:** Penggunaan *Dictionary* meniadakan waktu penelusuran saat proses identifikasi nama kontak, menjadikan proses CRUD efisien $O(1)$.
2. **Aman dari Data Hilang:** Memiliki fungsi penyimpanan berkas secara permanen (*File Handling Text*), sehingga seluruh modifikasi akan menetap.
3. **Kemudahan Pencarian:** Mengadopsi perbandingan *lower-case string*. Pengguna tidak diwajibkan menulis nama secara akurat, sistem bisa mencari kecocokan antar kata (*substring matching*).
4. **Sistem yang Solid:** Berbagai *edge-case* seperti input **key** kosong, menu tidak wajar, hingga data tidak ditemukan telah divalidasi, sehingga program tidak mudah **crash** karena *traceback errors*.

4.3 Keterbatasan Sistem

1. **Antarmuka Terbatas:** Masih berjalan di antarmuka konsol baris perintah (CLI), sehingga bagi orang awam terlihat kaku.
2. **Belum Database-Relational:** Penyimpanan menggunakan **plain text** dengan pemisahan koma, bukan sebuah sistem basis data DBMS (seperti MySQL), sehingga integritas kompleks tidak bisa dibangun.
3. **Manajemen Perubahan Simultan (Multiuser):** Program ini **single-state**, artinya perubahan pada satu sisi saja. Jika program dibuka secara bersamaan oleh dua instance (terminal berbeda), penyimpanan akan tumpang tindih.

BAB V. KENDALA DAN SOLUSI

Kendala dan Tantangan	Solusi yang Diterapkan
Kesulitan mengubah "Nama" pada fitur Edit Kontak, karena tipe data <i>Dictionary</i> secara fundamental menjadikan <i>key</i> (kunci/nama) bersifat <i>immutable</i> dan tidak bisa diubah langsung (<i>renaming key</i>).	Dilakukan manipulasi data memori tiga langkah: Nomor lama disimpan dalam variabel penyangga, fungsi bawaan python <code>del dictionary[key]</code> digunakan untuk membongkar pasangan lama, kemudian <i>key</i> nama baru dibuat dan disematkan dengan variabel nomor tadi.
Program mengalami <i>*crash*</i> saat dijalankan untuk pertama kalinya pada <i>environment</i> sistem lain karena ketiadaan berkas kontak.txt saat dipanggil fungsi baca 'r'.	Modul bawaan Python, yakni <code>os</code> diimpor untuk menjalankan kondisi pra-syarat <code>os.path.exists()</code> di dalam fungsi <code>muat_data()</code> . Jika berkas tidak ada, <i>*parsing*</i> file diabaikan.
Pencarian data ("Budi") gagal jika pengguna memasukkan kata kunci berupa karakter acak kapital-kecil ("buDI").	Seluruh teks <i>*input search*</i> dan teks <i>*database list*</i> dikonversi sementara dengan pemanggilan <i>method</i> <code>.lower()</code> sebelum dilakukan perbandingan <i>if-statement</i> .
Dictionary tidak memiliki urutan spesifik,	Data dari <i>Dictionary</i> diekstrak menjadi <i>List</i>

Kendala dan Tantangan	Solusi yang Diterapkan
sehingga proses pengurutan (Sorting) tidak bisa dilakukan langsung.	<i>of Tuples</i> menggunakan sintaks <code>list(data.items())</code> , kemudian diurutkan dengan algoritma Quick Sort.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

Nama Anggota	Tugas Utama	Kontribusi Detail
Ivan Farel Aditya	Struktur Program, Menu Main Loop & Tampilan Dasar	Menginisialisasi file <code>kelompok8.py</code> , merancang loop <i>while</i> pada antarmuka menu utama, merancang fungsi <code>tambah_kontak()</code> beserta validasi pencegah duplikat <i>key</i> , dan mendesain estetika <code>*print console*</code> untuk fungsi baca.
Raffan Fatih Zulfan	Manipulasi Struktur Data & Fitur Hapus/Edit	Menangani logika teknis fungsi <code>update_kontak()</code> dengan logika manipulasi <code>*key dictionary*</code> , membangun fitur <code>hapus_kontak()</code> , serta menangani error validasi <code>*edge cases*</code> jika entri nama tidak ditemukan.
Aulia Rezki Rahman	File Handling & Algoritma (Quick Sort & Search)	Mengembangkan implementasi <i>Quick Sort</i> (beserta rekursi dan partisi), pencarian teks <i>substring matching</i> di <code>cari_kontak()</code> , mengatur skenario sinkronisasi memori program dan menulis/membaca berkas

Nama Anggota	Tugas Utama	Kontribusi Detail
		plain-text menggunakan library OS.

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

- Proyek ini membuktikan bahwa penggunaan tipe data **Dictionary** untuk penyimpanan kontak sangat efektif, memberikan akses informasi instan ($O(1)$) yang bebas dari replikasi data yang identik.
- Algoritma *Quick Sort* dan *Sequential Substring Search* berhasil meminimalkan usaha komputasi pada penyortiran data abjad, menjadikan proses transisi *state* dari program menuju berkas fisik .txt berjalan mulus dan minim cacat.
- Konsep penanganan **file system** untuk mengimpor dan mengeksport variabel di python telah sempurna memastikan seluruh perubahan state operasional CRUD bersifat permanen lintas-sesi terminal.

Saran Pengembangan

- Antarmuka program diusulkan untuk berpindah dari terminal baris perintah menuju antarmuka grafis (*Graphical User Interface* - GUI) dengan pustaka seperti Tkinter atau PyQt.
- Penggunaan manajemen data bisa dikembangkan (*scale-up*) menggunakan bahasa kueri (SQL) dan basis data nyata seperti SQLite untuk menangani entri nomor ponsel berlapis dan relasional pada satu ID pengguna.

DAFTAR PUSTAKA

1. B. Miller and D. Ranum, *Problem Solving with Algorithms and Data Structures Using Python*, Franklin, Beedle & Associates, 2011.
2. M. T. Goodrich, R. Tamassia, M. Goldwasser, *Data Structures and Algorithms in Python*, Wiley, 2013.
3. Python Software Foundation, "The Python Language Reference," Python Documentation, 2024. [Online]. Available: <https://docs.python.org/3/reference/>.

LAMPIRAN

Lampiran A. Link GitHub

<https://github.com/raffanfatih/Project-Kelompok-8/tree/main>

Lampiran B. Screenshot Program

```
=== DATA BUKU TELEPON ===  
1. Tambah Kontak  
2. Lihat Kontak  
3. Cari Kontak  
4. Update Kontak  
5. Hapus Kontak  
6. Simpan Data Manual  
7. Keluar  
Pilih menu (1-7): █
```

Pilih menu (1-7): 1

--- TAMBAH KONTAK BARU ---

Masukkan Nama: budi

Masukkan Nomor Telepon: 081273893349

Kontak 'Budi' berhasil ditambah (Pilih menu 6 untuk simpan permanen!).

Pilih menu (1-7): 1

--- TAMBAH KONTAK BARU ---

Masukkan Nama: raffan

Eitsss! Kontak dengan nama 'Raffan' sudah ada.

Pilih menu (1-7): 2

--- DAFTAR KONTAK ---

Buku telepon masih kosong.

Pilih menu (1-7): 2

--- DAFTAR KONTAK ---

Nama: Aulia		Nomor: 08413393749
Nama: Ivan		Nomor: 0863548992
Nama: Raffan		Nomor: 08123455678
Nama: Raihan		Nomor: 0824832798203
Nama: Raxen		Nomor: 0863293723932
Nama: Budi		Nomor: 081273893349

Pilih menu (1-7): 3

--- CARI KONTAK ---

1. Cari berdasarkan Nama
2. Cari berdasarkan Nomor

Pilih metode pencarian (1/2): 1

Masukkan Nama yang dicari: ra

Ketemu! -> Nama: Raffan	Nomor: 08123455678
Ketemu! -> Nama: Raihan	Nomor: 0824832798203
Ketemu! -> Nama: Raxen	Nomor: 0863293723932

Pilih menu (1-7): 4

--- UPDATE KONTAK ---

Masukkan Nama kontak yang ingin diubah: raihan

1. Ubah Nama saja
2. Ubah Nomor saja

Pilih bagian yang ingin diubah (1/2): 1

Masukkan Nama Baru: randy

Nama 'Raihan' berhasil diubah menjadi 'Randy'.

Pilih menu (1-7): 5

--- HAPUS KONTAK ---

Masukkan Nama kontak yang ingin dihapus: raxen

Kontak 'Raxen' dihapus dari memori (Pilih menu 6 untuk simpan permanen!).

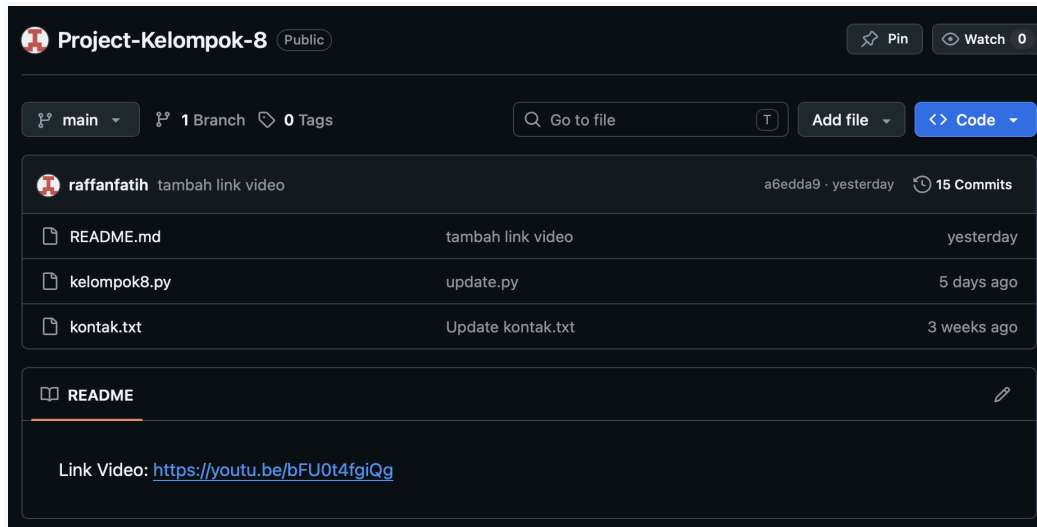
Pilih menu (1-7): 6

Mantap! Data berhasil diurutkan (A-Z) dan disimpan permanen ke kontak.txt.

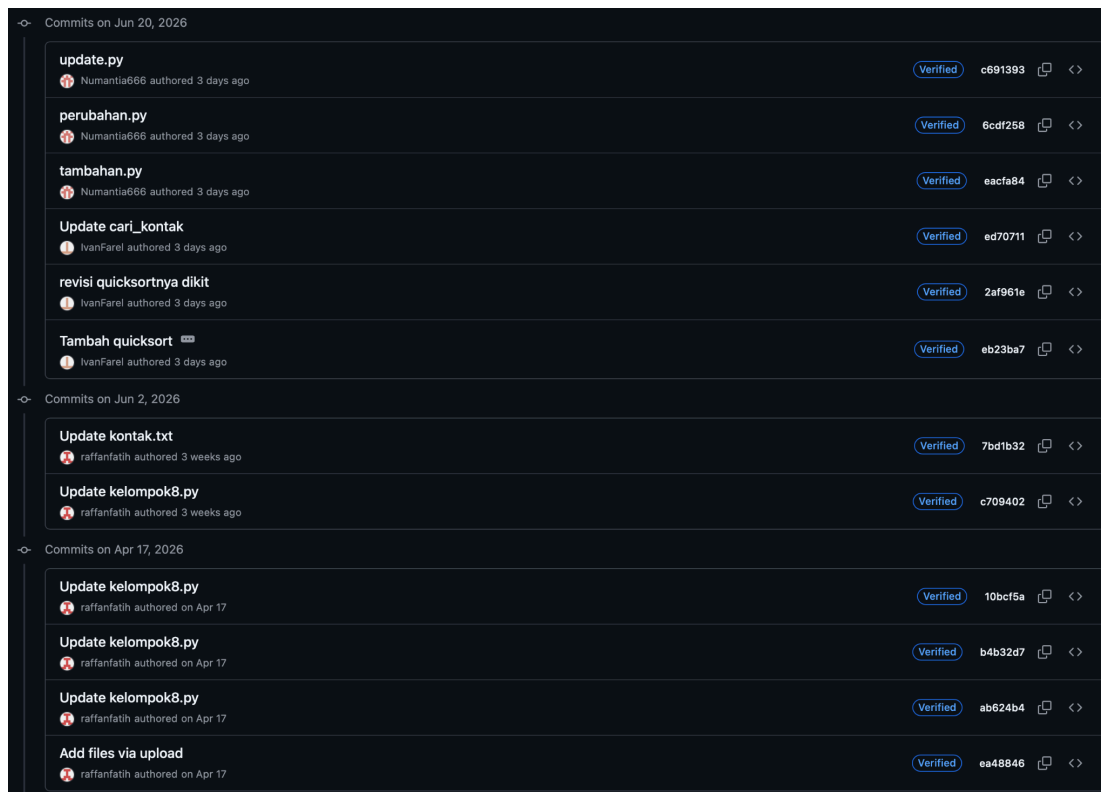
Pilih menu (1-7): 7

Keluar aplikasi. Pastikan kamu sudah menyimpan datamu!

Lampiran C. Struktur Folder Proyek



Lampiran D. Commit History GitHub



Lampiran E. Source Code Utama

Python

```
import os

# Nama file menggunakan ekstensi .txt untuk kemudahan membaca teks biasa
NAMA_FILE = 'kontak.txt'

# =====
# FUNGSI ALGORITMA PENGURUTAN (QUICK SORT)
# =====

def quickSort(data):
    # Memanggil fungsi dengan indeks awal 0 dan indeks akhir
    quickSortHelper(data, 0, len(data)-1)

def quickSortHelper(data, first, last):
    if first < last:
        # Mencari titik pisah atau splitpoint menggunakan fungsi partition
        splitpoint = partition(data, first, last)

        # Mengurutkan bagian kiri dari splitpoint secara rekursif
        quickSortHelper(data, first, splitpoint-1)
        # Mengurutkan bagian kanan dari splitpoint secara rekursif
        quickSortHelper(data, splitpoint+1, last)

def partition(data, first, last):
    # Patokan atau acuan diambil dari "Nama" pada elemen indeks pertama
    # data[first] adalah tuple ('Nama', 'Nomor'), jadi kita ambil [0]-nya
    acuan = data[first][0]

    leftmark = first + 1
    rightmark = last
    done = False

    while not done:
        # leftmark bergerak ke kanan selama namanya <= patokan
        while leftmark <= rightmark and data[leftmark][0] <= acuan:
```

```

        leftmark = leftmark + 1

    # rightmark bergerak ke kiri selama namanya >= patokan
    while data[rightmark][0] >= acuan and rightmark >= leftmark:
        rightmark = rightmark - 1

    # Jika kedua penunjuk saling melewati, partisi selesai
    if rightmark < leftmark:
        done = True
    else:
        # Swap elemen utuh (Nama beserta Nomor) di leftmark dan rightmark
        temp = data[leftmark]
        data[leftmark] = data[rightmark]
        data[rightmark] = temp

    # Kembalikan elemen acuan ke posisi tengah yang tepat di rightmark
    temp = data[first]
    data[first] = data[rightmark]
    data[rightmark] = temp

    return rightmark

# =====
# FUNGSI MANAJEMEN FILE TXT
# =====

def muat_data():
    data = {}
    # Mengecek apakah file .txt sudah ada sebelumnya
    if os.path.exists(NAMA_FILE):
        # Membuka file mode 'r' (read) untuk membaca data
        with open(NAMA_FILE, 'r') as file:
            for line in file:
                if ',' in line:
                    # Memisahkan baris teks menjadi 'nama' dan 'nomor'
                    # menggunakan koma
                    nama, nomor = line.strip().split(',', 1)
                    data[nama] = nomor

```

```

    return data

def simpan_data(data):
    list_kontak = list(data.items())

    # Memanggil fungsi quick sort dimana data list_kontak akan langsung
    terurut secara otomatis
    quickSort(list_kontak)

    with open(NAMA_FILE, 'w') as file:
        for nama, nomor in list_kontak:
            file.write(f"{nama},{nomor}\n")

    data.clear()
    for nama, nomor in list_kontak:
        data[nama] = nomor

    print("\nMantap! Data berhasil diurutkan (A-Z) dan disimpan permanen ke
kontak.txt.")

# =====
# APLIKASI CRUD: BUKU TELEPON PINTAR
# =====

# Memuat data ke memori saat program pertama kali dijalankan
daftar = muat_data()

def tambah_kontak():
    print("\n--- TAMBAH KONTAK BARU ---")
    # .title() memastikan huruf pertama setiap kata kapital (misal: "budi" ->
    "Budi")
    nama = input("Masukkan Nama: ").title()

    if nama in daftar:
        print(f"Eitsss! Kontak dengan nama '{nama}' sudah ada.")
    else:
        nomor = input("Masukkan Nomor Telepon: ")
        # Menambahkan data baru (key: nama, value: nomor) ke dalam dictionary

```

```

        daftar[nama] = nomor
        print(f"Kontak '{nama}' berhasil ditambah (Pilih menu 6 untuk simpan
permanen!).")

def lihat_kontak():
    print("\n--- DAFTAR KONTAK ---")
    if not daftar:
        print("Buku telepon masih kosong.")
    else:
        # Looping untuk menampilkan seluruh isi dictionary
        for nama, nomor in daftar.items():
            print(f" Nama: {nama} \t| Nomor: {nomor}")

def cari_kontak():
    print("\n--- CARI KONTAK ---")
    print("1. Cari berdasarkan Nama")
    print("2. Cari berdasarkan Nomor")
    opsi = input("Pilih metode pencarian (1/2): ")

    ditemukan = False

    if opsi == '1':
        # Mengubah input ke title case untuk mencocokkan dengan format data
        kata_kunci = input("Masukkan Nama yang dicari: ").title()
        for nama, nomor in daftar.items():
            # Pencarian substring (mencari apakah kata kunci ada di dalam nama
kontak)
            if kata_kunci.lower() in nama.lower():
                print(f" Ketemu! -> Nama: {nama} \t| Nomor: {nomor}")
                ditemukan = True

    elif opsi == '2':
        kata_kunci = input("Masukkan Nomor yang dicari: ")
        for nama, nomor in daftar.items():
            if kata_kunci in nomor:
                print(f" Ketemu! -> Nama: {nama} \t| Nomor: {nomor}")
                ditemukan = True

```



```

else:
    print("Pilihan tidak valid.")
    return

if not ditemukan:
    print("Tidak ada kontak yang cocok dengan pencarianmu.")

def update_kontak():
    print("\n--- UPDATE KONTAK ---")
    nama_lama = input("Masukkan Nama kontak yang ingin diubah: ").title()

    # Mengecek apakah kontak yang ingin diubah benar-benar ada di memori
    if nama_lama in daftar:
        print("1. Ubah Nama saja")
        print("2. Ubah Nomor saja")
        opsi = input("Pilih bagian yang ingin diubah (1/2): ")

        if opsi == '1':
            nama_baru = input("Masukkan Nama Baru: ").title()
            if nama_baru in daftar:
                print(f>Nama '{nama_baru}' sudah digunakan kontak lain.")
            else:
                # Menyimpan nomor lama, menghapus key lama, dan membuat key
baru
                nomor_lama = daftar[nama_lama]
                del daftar[nama_lama]
                daftar[nama_baru] = nomor_lama
                print(f" Nama '{nama_lama}' berhasil diubah menjadi
'{nama_baru}'.")

        elif opsi == '2':
            nomor_baru = input("Masukkan Nomor Telepon Baru: ")
            # Meng-overwrite (menimpa) value lama dengan value baru
            daftar[nama_lama] = nomor_baru
            print(f"Nomor untuk '{nama_lama}' berhasil diperbarui.")

        else:
            print("Pilihan tidak valid.")

```

```

    else:
        print(f" Kontak '{nama_lama}' tidak ditemukan.")

def hapus_kontak():
    print("\n--- HAPUS KONTAK ---")
    nama = input("Masukkan Nama kontak yang ingin dihapus: ").title()

    if nama in daftar:
        # Menghapus key dan value dari dictionary
        del daftar[nama]
        print(f"Kontak '{nama}' dihapus dari memori (Pilih menu 6 untuk simpan
permanen!).")
    else:
        print(f" Kontak '{nama}' tidak ditemukan.")

# =====
# PROGRAM UTAMA (MAIN LOOP)
# =====

def main():
    # Looping tak terbatas sampai dipatahkan oleh perintah 'break'
    while True:
        print("\n=== DATA BUKU TELEPON ===")
        print("1. Tambah Kontak")
        print("2. Lihat Kontak")
        print("3. Cari Kontak")
        print("4. Update Kontak")
        print("5. Hapus Kontak")
        print("6. Simpan Data Manual")
        print("7. Keluar")

        pilihan = input("Pilih menu (1-7): ")

        if pilihan == '1':
            tambah_kontak()
        elif pilihan == '2':
            lihat_kontak()

```

```
elif pilihan == '3':
    cari_kontak()
elif pilihan == '4':
    update_kontak()
elif pilihan == '5':
    hapus_kontak()
elif pilihan == '6':
    simpan_data(daftar)
elif pilihan == '7':
    print(" Keluar aplikasi. Pastikan kamu sudah menyimpan datamu!")
    # Menghentikan loop dan keluar dari program
    break
else:
    print("Pilihan tidak valid! Silakan pilih angka 1-7.")

# Menjalankan fungsi main() hanya jika script dieksekusi langsung
if __name__ == "__main__":
    main()
```